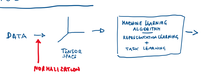
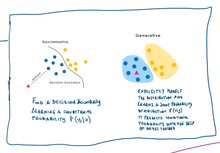
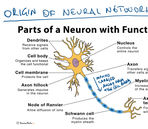
# PIPELINE IN ML




## GENERATIVE VS. DISCRIMINATIVE MODELS




## ORIGIN OF NEURAL NETWORKS AND DEEP LEARNING




SUM

**1958** — PERCEPTRON (ROSENBLATT)
RESEMBLE A NEURON
- ADJUST PARAMETERS WHETH DATA $\in \mathbb{R}^d$
- METHOD TO LEARN ADJUST PARAMETERS

**1960** — ADALINE (WIDROW AND HOFF)
ADAPTIVE LINEAR NEURON PHYSICAL DEVICE
ADJUST PARAMETERS WITH WEIGHTED SUM OF THE INPUT

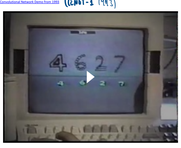
**1969** — PERCEPTRONS (MINSKY AND PAPERT)
MATHEMATICAL PROOF OF CONVERGENCE AND LIMITATIONS (E.G. XOR FUNCTION + CONVEXITY + MLP)

**AI WINTER**

**1970-1986** — BACK PROPAGATION (LINNAINMAA 1970) (WERBOS 1974) (RUMELHART, HINTON, WILLIAMS 1986)
COMPUTATIONAL METHODS USEFUL TO LEARN MLP

**1997** — LSTM (HOCHREITER, SCHMIDHUBER)
TEMPORAL DATA, IMAGES SEQUENCES

**1998** — CNN (LECUN) (LECUN BOTTOU BENGIO HAFFNER)
FOR OCR
Convolutional Network Demos from 1989 (LECUN-E 1993)




**2006** — DEEP LEARNING (HINTON, OSINDERO, TEH)
- DEEP BELIEF NETS
- UNSUPERVISED LAYER BY LAYER
- TOP DOWN SUPERVISED TUNING
- REVIVAL OF NEURAL NETWORKS

**2009** — IMAGENET (DENG, FEI-FEI)
Data set of labeled images, 500 images/class, available for research uses

**2015** — ALEXNET (KRIZHEVSKY, SUTSKEVER, HINTON)
An unknown prof. No. Grads in CMU
Trained on GPUs
"Dropout, data augmentation, ReLU, stochastic gradient descent with momentum"
ONE WINNER MANY RUNNER UPS
Linear, Volume -1

**DEEP LEARNING EXPLOSION ERA**

**2017** — KERAS (CHOLLET)

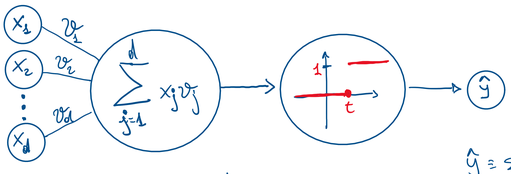
**2016** — TENSOR FLOW (GOOGLE)

**ALL PREVIOUS AND TRAINED LIBRARIES ALREADY**
Any student/scientist needs to have a look to ecosystem
TENSORFLOW
- KERAS, PYT.
- SCIKIT-LEARN
- KERAS, MATPLOTLIB
- MATPLOTLIB, PYTORCH

## PERCEPTRON (LIBRO: NEURAL NETWORK - A SYSTEMATIC INTRODUCTION, CAP3)

PROPOSED BY ROSENBLATT AS MACHINE FOR BINARY CLASSIFICATION $\Rightarrow x \in \mathbb{R}^d, y \in \{0,1\}$

MAIN IDEA
- ONE WEIGHT $w_j$ $\forall$ INPUT COMPONENT $x_j$.
- MULTIPLY WEIGHTS WITH RESPECT TO THE INPUTS AND SUM ALL OF THEM (LINEAR COMBINATION)
- IF RESULT IS LARGER THAN A THRESHOLD $t$, THAN RETURN 1 OTHERWISE RETURN 0.



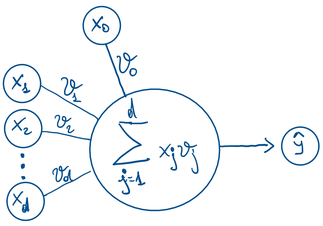

$\hat{y}$ = STIMA

$y$ = ETICHETTA DEL TRAINING

$$\hat{y} = \begin{cases} 1 & \sum_{j=1}^{d} x_j w_j > t \\ 0 & \text{OTHERWISE} \end{cases}$$

↑ PARAMETRIZED SIGN FUNCTION

How to set threshold?
- ADD BIAS $x_0 = +1$ AND $w_0$ IN BRAIN TO LEARN ALSO THE THRESHOLD




WE HAVE A MODEL. HOW TO TRAIN IT? HOW TO ADJUST PARAMETERS?

## PERCEPTRON TRAINING (LIBRO: NEURAL NETWORK - A SYSTEMATIC INTRODUCTION, CAP4)

ROSENBLATT MAIN INNOVATION WAS ON THE TRAINING PROCEDURES FOR PERCEPTRON

### ALGORITHM FOR LEARNING PERCEPTRON

1) INITIALIZE WEIGHTS RANDOMLY WITH SMALL VALUES $\in [0,1]$ AND SET RANDOM THRESHOLD $\in \mathbb{R}$

$w_j \leftarrow$ RANDOM $([0,1])$

$t \leftarrow$ RANDOM $(\mathbb{R})$

2) TAKE SAMPLES $x^{(i)}$ AND PREDICT $\hat{y}^{(i)}$ BY USING PERCEPTRON MODEL

3) FOR ERRONEOUS PREDICTIONS UPDATE WEIGHTS AS FOLLOWS:
- IF THE PREDICTION $\hat{y}^{(i)} = 0$ AND $y^{(i)} = 1$ THEN "INCREASE WEIGHTS"
- IF THE PREDICTION $\hat{y}^{(i)} = 1$ AND $y^{(i)} = 0$ THEN "DECREASE WEIGHTS"

4) REPEAT FROM 2 UNTIL "NO ERRORS ARE MADE"

How to write this with math?

$$w_j \leftarrow w_j + \left(y^{(i)} - \hat{y}^{(i)}\right) x_j^{(i)}$$

- $w_j$: new $w_j$ at time $t+1$; current value at time $t$
- $\left(y^{(i)} - \hat{y}^{(i)}\right)$: THESE CAN ASSUME VALUES
- $x_j^{(i)}$: WE CAN ASSUME NORMALIZATION IN $[0,1]$; OR WE CAN SUBSTITUTE WITH A SMALL $\eta \in [0,1]$

| $y^{(i)}$ | $\hat{y}^{(i)}$ |
|---|---|
| 0 | 0 |
| 0 | 1 |
| 1 | 0 |
| 1 | 1 |

IS THE ABOVE PROCEDURE CONVERGING?
WHO IS T?
WHO IS M?
WHICH IS THE MODEL?
WHO IS P?
HOW TO EVALUATE ERRORS DURING TRAINING?

CASE OF PERCEPTRON

COST FUNCTION (LOSS FUNCTION) $\equiv$ COUNT # OF ERRORS $\hat{y}^{(i)} \neq y^{(i)}$ WITH RESPECT TO THE TRAINING SET.

↑ WE WISH TO MINIMIZE

## ADALINE ALGORITHM VARIATION

WEIGHTS CAN BE ADJUSTED WITH A WEIGHTED SUM OF THE INPUT $\hat{y} = \sum_j w_j x_j$